

services\data\nvd_downloader.py

```
# Operating system interface for file and directory operations
import os
# JSON processing for vulnerability data handling
import json
# Gzip compression for efficient storage of large datasets
import gzip
# HTTP client library for NVD API communication
import requests
# Type annotations for function parameters and return values
from typing import List, Dict
# Application configuration for API credentials and paths
import config

class NVDDownloader:
    """Download and manage historical NVD data feeds"""

    def __init__(self):
        # Updated to use the correct NVD 2.0 API endpoints for historical data
        self.base_url = "https://services.nvd.nist.gov/rest/json/cves/2.0"
        # Load historical data directory from configuration
        self.historical_dir = config.NVD_HISTORICAL_DIR
        # Set generous timeout for large dataset downloads (5 minutes)
        self.timeout = 300
        # Load API key for authenticated access with higher rate limits
        self.api_key = config.NVD_API_KEY

    def download_year_via_api(self, year: int, save_as_json: bool = True) -> bool:
        """Download CVE data for a specific year using NVD API 2.0"""
        # Choose file extension based on format preference
        file_extension = ".json" if save_as_json else ".json.gz"
        filepath = os.path.join(self.historical_dir, f"CVE-{year}{file_extension}")

        # Skip download if file already exists
        if os.path.exists(filepath):
            print(f"[Downloader] {year} data already exists, skipping")
            return True

        try:
            print(f"[Downloader] Downloading {year} data via API...")
            # Use API 2.0 with full year date range for comprehensive coverage
            params = {
                "resultsPerPage": 2000, # Large page size for efficiency
                "startIndex": 0,
                "pubStartDate": f"{year}-01-01T00:00:00.000",
                "pubEndDate": f"{year}-12-31T23:59:59.999"
            }

            # Setup authentication headers if API key available
            headers = {"apiKey": self.api_key} if self.api_key else {}
            all_cves = []

            # Handle pagination to retrieve complete dataset
            while True:
                response = requests.get(
                    self.base_url,
                    params=params,
                    headers=headers,
                    timeout=self.timeout
                )

                if response.status_code == 200:
                    data = response.json()
                    vulnerabilities = data.get("vulnerabilities", [])

                    # Break if no more data available
                    if not vulnerabilities:
                        break

                    # Accumulate CVE data from this page
                    all_cves.extend(vulnerabilities)

            # Check if there are more results to fetch
```

```

total_results = data.get("totalResults", 0)
if len(all_cves) >= total_results:
    break

# Update start index for next page
params["startIndex"] += params["resultsPerPage"]

elif response.status_code == 403:
    print(f"[Downloader] API access forbidden for {year}")
    return False
else:
    print(f"[Downloader] Failed to download {year}: {response.status_code}")
    return False

# Prepare data for saving in NVD JSON 2.0 format
data_to_save = {"vulnerabilities": all_cves}

if save_as_json:
    # Save as plain JSON for fast access
    with open(filepath, 'w', encoding='utf-8') as f:
        json.dump(data_to_save, f, indent=2)
    print(f"[Downloader] Downloaded {year} successfully ({len(all_cves)} CVEs) - saved as JSON")
else:
    # Save as compressed JSON for storage efficiency
    with gzip.open(filepath, 'wt', encoding='utf-8') as f:
        json.dump(data_to_save, f, indent=2)
    print(f"[Downloader] Downloaded {year} successfully ({len(all_cves)} CVEs) - saved as compressed JSON")

return True

except Exception as e:
    print(f"[Downloader] Error downloading {year}: {e}")
    return False

def download_year(self, year: int, save_as_json: bool = True) -> bool:
    """Download CVE data for a specific year"""
    # Simple wrapper for API download method
    return self.download_year_via_api(year, save_as_json)

def download_all_years(self, years: List[int] = None, save_as_json: bool = True) -> Dict[int, bool]:
    """Download data for multiple years"""
    # Use configuration default years if none specified
    if years is None:
        years = config.HISTORICAL_YEARS

    # Track success/failure for each year
    results = {}
    for year in years:
        results[year] = self.download_year(year, save_as_json)

    # Report overall operation success
    successful = sum(1 for success in results.values() if success)
    print(f"[Downloader] Downloaded {successful}/{len(years)} years successfully")
    return results

def is_year_available(self, year: int) -> bool:
    """Check if year data is downloaded (checks both JSON and GZ formats)"""
    # Check for both possible file formats
    json_filepath = os.path.join(self.historical_dir, f"CVE-{year}.json")
    gz_filepath = os.path.join(self.historical_dir, f"CVE-{year}.json.gz")
    return os.path.exists(json_filepath) or os.path.exists(gz_filepath)

def get_available_years(self) -> List[int]:
    """Get list of years with downloaded data"""
    available = []
    # Check each configured historical year for data availability
    for year in config.HISTORICAL_YEARS:
        if self.is_year_available(year):
            available.append(year)
    return sorted(available)

def convert_gz_to_json(self, year: int) -> bool:
    """Convert existing .gz file to plain .json file"""
    gz_filepath = os.path.join(self.historical_dir, f"CVE-{year}.json.gz")

```

```

json_filepath = os.path.join(self.historical_dir, f"CVE-{year}.json")

# Check if source file exists
if not os.path.exists(gz_filepath):
    print(f"[Downloader] No .gz file found for {year}")
    return False

# Skip if target already exists
if os.path.exists(json_filepath):
    print(f"[Downloader] JSON file already exists for {year}")
    return True

try:
    print(f"[Downloader] Converting {year} from .gz to .json...")
    # Read compressed file and write as plain JSON
    with gzip.open(gz_filepath, 'rt', encoding='utf-8') as gz_file:
        data = json.load(gz_file)

    with open(json_filepath, 'w', encoding='utf-8') as json_file:
        json.dump(data, json_file, indent=2) # Pretty-print for readability

    print(f"[Downloader] Successfully converted {year} to JSON format")
    return True

except Exception as e:
    print(f"[Downloader] Error converting {year}: {e}")
    return False

def convert_all_gz_to_json(self) -> Dict[int, bool]:
    """Convert all existing .gz files to .json files"""
    results = {}
    # Convert each configured historical year
    for year in config.HISTORICAL_YEARS:
        results[year] = self.convert_gz_to_json(year)

    # Report overall conversion success
    successful = sum(1 for success in results.values() if success)
    print(f"[Downloader] Converted {successful}/{len(results)} files successfully")
    return results

```